HDOC Day-41 Activity

Menu-Driven C Program - Operations on Two Paragraphs

Name	Gurveer Singh Cheema
SAF ID	590032925
Batch	15

Aim

To write a menu-driven C program that reads two paragraphs and performs string-processing operations using separate user-defined functions. The program uses pointers/arrays wherever data is modified or multiple results are produced.

Header Files Used

stdio.h for input/output, **string.h** for strtok(), strlen(), strcpy(), and strcmp(), and **ctype.h** for isalpha(), isspace(), tolower(), and toupper().

Operations

1. Read two paragraphs.
2. Find the longest and shortest words in each paragraph.
3. Remove repeated consecutive spaces.
4. Convert each paragraph to title case.
5. Find the frequency of every distinct word without considering letter case.
6. Exit.

Algorithm

1. Read two paragraphs using fgets().
2. Display the menu repeatedly using a do-while loop.
3. Use switch-case to perform the selected operation on both paragraphs.
4. For longest/shortest words, tokenize a copy of the paragraph and compare word lengths.
5. Remove repeated spaces by scanning character by character.
6. Convert the first alphabetic character of each word to uppercase and the remaining letters to lowercase.
7. For word frequencies, convert words to lowercase, store distinct words, and count repetitions.
8. Stop when the user selects Exit.

C Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 1000
#define MAX_WORDS 200
#define WORD_LEN 100

void readParagraph(char paragraph[], size_t size)
{
    fgets(paragraph, size, stdin);
    paragraph[strcspn(paragraph, "\n")] = '\0';
}

void findLongestAndShortestWords(const char paragraph[],
                                 char longestWord[],
                                 char shortestWord[])
{
```

```
    char copy[MAX];
    char *word;

    strcpy(copy, paragraph);
    word = strtok(copy, " \t,.;:!?\"()[]{}-");

    if (word == NULL)
    {
        strcpy(longestWord, "");
        strcpy(shortestWord, "");
        return;
    }

    strcpy(longestWord, word);
    strcpy(shortestWord, word);

    while (word != NULL)
    {
        if (strlen(word) > strlen(longestWord))
            strcpy(longestWord, word);

        if (strlen(word) < strlen(shortestWord))
            strcpy(shortestWord, word);

        word = strtok(NULL, " \t,.;:!?\"()[]{}-");
    }
}

void removeRepeatedSpaces(const char paragraph[],
                          char modifiedParagraph[])
{
    int i = 0, j = 0;
    int previousSpace = 0;

    while (paragraph[i] != '\0')
    {
        if (isspace((unsigned char)paragraph[i]))
        {
            if (!previousSpace)
            {
                modifiedParagraph[j++] = ' ';
                previousSpace = 1;
            }
        }
        else
        {
            modifiedParagraph[j++] = paragraph[i];
            previousSpace = 0;
        }
        i++;
    }

    modifiedParagraph[j] = '\0';
}

void convertToTitleCase(const char paragraph[],
                        char titleCaseParagraph[])
{
    int i;
    int newWord = 1;

    for (i = 0; paragraph[i] != '\0'; i++)
    {
        unsigned char ch = (unsigned char)paragraph[i];

        if (isalpha(ch))
        {
            if (newWord)
                titleCaseParagraph[i] = toupper(ch);
            else
                titleCaseParagraph[i] = tolower(ch);
            newWord = 0;
        }
        else
        {
            titleCaseParagraph[i] = paragraph[i];
            if (isspace(ch))
                newWord = 1;
        }
    }

    titleCaseParagraph[i] = '\0';
}

void toLowerWord(char word[])
{
    int i;
    for (i = 0; word[i] != '\0'; i++)
        word[i] = tolower((unsigned char)word[i]);
}
void findWordFrequencies(const char paragraph[])
```

```
{
    char copy[MAX];
    char words[MAX_WORDS][WORD_LEN];
    int frequency[MAX_WORDS] = {0};
    int count = 0, i, found;
    char *word;

    strcpy(copy, paragraph);
    word = strtok(copy, " \t,.;:!?\"()[]{}-");

    while (word != NULL && count < MAX_WORDS)
    {
        char current[WORD_LEN];
        strncpy(current, word, WORD_LEN - 1);
        current[WORD_LEN - 1] = '\0';
        toLowerWord(current);

        found = -1;
        for (i = 0; i < count; i++)
        {
            if (strcmp(words[i], current) == 0)
            {
                found = i;
                break;
            }
        }

        if (found >= 0)
            frequency[found]++;
        else
        {
            strcpy(words[count], current);
            frequency[count] = 1;
            count++;
        }

        word = strtok(NULL, " \t,.;:!?\"()[]{}-");
    }

    if (count == 0)
    {
        printf("No words found.\n");
        return;
    }

    for (i = 0; i < count; i++)
        printf("%s : %d\n", words[i], frequency[i]);
}

int main()
{
    char p1[MAX], p2[MAX];
    char longest[WORD_LEN], shortest[WORD_LEN];
    char modified[MAX], titleCase[MAX];
    int choice;

    printf("Enter Paragraph 1: ");
    readParagraph(p1, sizeof(p1));

    printf("Enter Paragraph 2: ");
    readParagraph(p2, sizeof(p2));

    do
    {
        printf("\n----- MENU -----\n");
        printf("1. Find longest and shortest words\n");
        printf("2. Remove repeated spaces\n");
        printf("3. Convert paragraphs to title case\n");
        printf("4. Find frequency of distinct words\n");
        printf("5. Display original paragraphs\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                findLongestAndShortestWords(p1, longest, shortest);
                printf("\nParagraph 1\nLongest: %s\nShortest: %s\n",
                       longest, shortest);

                findLongestAndShortestWords(p2, longest, shortest);
                printf("Paragraph 2\nLongest: %s\nShortest: %s\n",
                       longest, shortest);
                break;

            case 2:
                removeRepeatedSpaces(p1, modified);
                printf("Paragraph 1: %s\n", modified);
                removeRepeatedSpaces(p2, modified);
                printf("Paragraph 2: %s\n", modified);
```

```
                break;

            case 3:
                convertToTitleCase(p1, titleCase);
                printf("Paragraph 1: %s\n", titleCase);
                convertToTitleCase(p2, titleCase);
                printf("Paragraph 2: %s\n", titleCase);
                break;

            case 4:
                printf("\nParagraph 1 word frequencies:\n");
                findWordFrequencies(p1);
                printf("\nParagraph 2 word frequencies:\n");
                findWordFrequencies(p2);
                break;

            case 5:
                printf("Paragraph 1: %s\n", p1);
                printf("Paragraph 2: %s\n", p2);
                break;

            case 6:
                printf("Exiting program...\n");
                break;

            default:
                printf("Invalid choice! Try again.\n");
        }

    } while (choice != 6);

    return 0;
}
```

Sample Input and Output

```
Enter Paragraph 1: C programming is very useful. C is powerful.
Enter Paragraph 2: strings   are easy to learn and strings are useful.

----- MENU -----
1. Find longest and shortest words
2. Remove repeated spaces
3. Convert paragraphs to title case
4. Find frequency of distinct words
5. Display original paragraphs
6. Exit
Enter your choice: 1

Paragraph 1
Longest: programming
Shortest: C
Paragraph 2
Longest: strings
Shortest: to

Enter your choice: 2
Paragraph 1: C programming is very useful. C is powerful.
Paragraph 2: strings are easy to learn and strings are useful.

Enter your choice: 3
Paragraph 1: C Programming Is Very Useful. C Is Powerful.
Paragraph 2: Strings   Are Easy To Learn And Strings Are Useful.

Enter your choice: 4
Paragraph 1 word frequencies:
c : 2
programming : 1
is : 2
very : 1
useful : 1
powerful : 1

Paragraph 2 word frequencies:
strings : 2
are : 2
easy : 1
to : 1
learn : 1
and : 1
useful : 1

Enter your choice: 6
Exiting program...
```

Function Explanation

readParagraph() reads a complete line with spaces using fgets(). **findLongestAndShortestWords()** makes a copy, separates it into words using strtok(), and compares their lengths. **removeRepeatedSpaces()** scans each character and keeps only one space when several spaces occur consecutively.

convertToTitleCase() capitalizes the first letter of each word and converts the remaining alphabetic characters to lowercase. **findWordFrequencies()** converts words to lowercase before comparing them, so words such as C and c are treated as the same word.

Important Concepts

strtok() splits a string into tokens. **strlen()** gives string length. **strcpy()** copies strings. **strcmp()** compares strings. **isspace()** detects whitespace. **tolower()** and **toupper()** change letter case.

Result

Thus, a menu-driven C program was successfully implemented to process two paragraphs using separate user-defined functions and standard string/character handling functions.

Quick Viva Questions

1. Why is a copy made before strtok()? Because strtok() modifies the string it tokenizes.
2. How are word frequencies made case-insensitive? Every word is converted to lowercase before

comparison.

3. What does isspace() check? It checks whitespace characters such as spaces and tabs.

4. What is title case? The first letter of each word is uppercase and the remaining letters are lowercase.

5. Why use fgets() for paragraphs? It reads a complete line including spaces.

6. What does strcmp() return for equal strings? It returns 0.